

Research Article

A Hybrid Deep Reinforcement Learning Approach for Algorithmic Trading in Commodity Futures Markets

Baljinder Kaur , Brahmaleen K. Sidhu , and Gurjit Singh Bhathal 

Department of Computer Science and Engineering, Punjabi University, Patiala, India

Correspondence should be addressed to Baljinder Kaur; baljinderkaur1057@gmail.com

Received 17 September 2025; Revised 14 November 2025; Accepted 15 November 2025

Guest Editor: Dr. Mukesh Kumar

Copyright © 2025 Baljinder Kaur et al. Applied Computational Intelligence and Soft Computing published by John Wiley & Sons Ltd. This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

The financial markets have been affected extensively in recent years, due to algorithmic trading, predominantly concerning commodity futures. This research provides a novel hybrid approach combining classical machine learning algorithms (random forest) with modern deep reinforcement learning algorithms (deep Q-network, proximal policy optimization, and soft actor-critic) for algorithmic trading in the commodity futures market. In contrast to prior investigations which mostly focus on equities, we applied this hybrid model to precious metals (gold) futures using a novel dataset of gold prices augmented with geopolitical risk indices, resulting in outstanding performance on financial risk metrics. The evaluation done suggests that the soft actor-critic agent modeled outshines conventional approaches (buy-and-hold and sell-and-hold) along with deep Q-network and proximal policy optimization, in relation to cumulative profit, Sharpe ratio, maximum drawdown, and annualized volatility. Our results enhance the algorithmic trading methodology growth by supporting problem reduction commonly associated with classical trading techniques or approaches. This study provides significant value for traders and financial institutions intending to implement artificial intelligence in their cognitive trading strategies.

Keywords: algorithmic trading; deep Q-network; deep reinforcement learning; proximal policy optimization; soft actor-critic

1. Introduction

Financial markets have changed as a consequence of the development of algorithmic trading, allowing traders to make judgments more quickly based on data. Earlier conventional trading algorithms faced both nonstationarity and volatility issues in commodity futures markets. To work upon these problems, deep reinforcement learning (DRL) helped in optimizing trading strategies by blending two strategies: reinforcement learning (RL) and neural networks (NNs). Commodity futures markets have factors like liquidity, volatility, and diversity, and thus, from all the assets, these are mostly considered in algorithmic trading. DRL is a combination of deep learning (DL) which inculcates NN to simulate complex patterns along with RL, a framework which trains an agent to pass judgments based on learning and its interaction with the environment to get improved

rewards, better returns, and fewer risks which includes determining the best price, timing, and volume. Since DRL can handle complexities in the futures market along with its dynamic nature, it provides a great motivation for its use in commodity futures trading. The main issue was the adaptability of traditional methods in dynamic markets since they follow a predefined rule book. On the contrary, DRL is capable of learning from new and raw data, thus making refined strategies resulting in optimized performance. Both financial institutions and traders can gain a competitive edge by implementing DRL and thus can have refined rewards, polished returns, and slighter risks.

This study proposed the first hybrid study of random forest (RF) + DRL (soft actor-critic (SAC), deep Q-network (DQN), proximal policy optimization (PPO)) for gold futures, inculcating geopolitical risk indices, which focuses on creating an algorithmic trading system in the commodity

futures market using DRL. The system is trained and tested using market data, thus giving maximum returns. Earlier studies mostly focused on the forex market and equities, with a few supporting studies showcasing its effectiveness in the commodity market. However, in most of the scenarios, the trading system using DRL relies only on DL without considering conventional ML model predictive techniques. This research covers all the potential research gaps including limited research on hybrid DRL approaches, more studies focused on stock markets instead of commodity futures, back-testing limitations, and a few more, which are highlighted after the literature review section. Thus, we proposed a novel hybrid framework that integrates RF regression for price forecasting with DRL agents (DQN, PPO, and SAC) to optimize trading strategies for gold futures. Thus, it offers a promising avenue for acquiring high trading performance. Not only is the framework novel, but the dataset integrated is novel as well. The proposed research is twofold: (1) The environment simulating historical market conditions and evaluating trading actions based on gold futures and (2) the agent uses DRL algorithms (DQN, PPO, and SAC) for processing market data and executing trades. The fundamental objective of the agent is to increase total rewards based on learned patterns by making informed buy, sell, or hold decisions. Therefore, we have suggested an action set which updates the trading rules automatically, along with this it allows the agent to learn to regulate positions, e.g., buy, sell, and hold actions. Furthermore, the reward statements train the agent for maximum profit by making informed trading decisions, buying low, and selling high while avoiding unnecessary holding of assets, resulting in outstanding results. DQN, PPO, and SAC agents have been trained, and performances have been compared with benchmark traditional methods buy and hold (B&H) and sell and hold (S&H), and results show that SAC agents outperformed. Thus, we can suggest that this research adds to the emerging domain of AI-powered trading by portraying the effectiveness of hybrid DRL frameworks in commodity futures markets. Along with this, our findings contribute worthy inputs for traders and financial institutions to improve their approaches for algorithmic trading with RL.

The research work is divided as follows: Section 2 evaluates existing literature and research gaps, Section 3 discusses problem formulation and methodology, Section 4 provides the experimental results and its analysis, and Section 5 discusses conclusions and future research directions.

2. Literature Review

This study gives an outline of the fundamental concepts, methodologies, and research findings relevant to our research. Algorithmic trading has been the subject of extensive investigation. They all look for unique solutions to improve the algorithm. This section will review popular literature based on its methodology. Traditional algorithmic trading systems utilize various strategies, including momentum trading, mean reversion, and arbitrage. Technical analysis is

the basis of these systems. For trading decisions, they rely on historical data and technical indicators. Because of their static nature and dependency on historical patterns, the ability of these systems is limited toward the dynamic nature of the market.

Current studies have gradually adopted machine learning (ML), DL, and DRL methods to overcome these boundaries. A numerical vector trend predicting method for predicting the daily spot price of Brent crude oil, outperforming traditional models such as ARIMA, SVR, and wavelet analysis models, was proposed by [1], while a new stochastic distribution, skewed Student's *t*-distribution, for silver, copper, and gold time-series estimation, which accounts for the time-dependent parameters and non-Gaussian behavior of time-series data, was proposed by [2, 3]. Reference [4, 5] employed Bayesian symbolic regression for crude oil price forecasting, emphasizing the incompetence of traditional econometric models such as ARDL, VAR, and ARIMA in managing nonlinear and nonstationary market data.

In predicting the prices of gold, [6] proposed an ARDL model with macroeconomic predictors; on the contrary, [7] used a deep belief network incorporating macroeconomic indices. Both [8–11] exhibited the proficiency of SVR in modeling nonlinear relationships which were subsequently utilized for gold and Bitcoin prediction, used LSTM for predicting gold prices, and [12] considered a CNN–LSTM hybrid approach to forecast gold volatility.

Few studies have utilized an ensemble approach for achieving improved results of forecasting. References [13, 14] proposed hybrid decomposition–ensemble frameworks combining SVM, XGBoost, and LSTM for the prediction of crude oil prices. Researchers upgraded this approach with secondary decomposition–reconstruction models [15]. Research work done by [16, 17] validated that RF attains strong predictive performance for gold, while [18] highlighted its significance for economic variable forecasting. Reference [19] combined a multilayer perceptron with whale optimization for gold prediction, outperforming ARIMA.

A DRL period came stronger than expected, which developed as a strong method for constructing adaptive trading systems. DRL algorithms such as DQN, PPO, and SAC have demonstrated superior adaptability and profitability. Reference [20] studied and surveyed DQN applications in trading, while [21] outlined theoretical framework for PPO and SAC. References [22, 23] examined DRL frameworks for commodity and financial trading. Although DRL has been underexplored in nonequity markets, its application is growing. Studies, including [24–28], exhibited that DRL-based trading systems outperform benchmark models return and risk management. In recent work, [29] proposed a GRU-based DDQN multiagent system for gold futures trading, achieving high performance in return, risk-adjusted metrics, and risk.

In general, the reviewed studies create a clear progression from traditional statistical models to advanced ML, DL, and DRL methods, highlighting the latter's capability to capture nonlinear market dynamics and enhancing

predictive accuracy in commodity price forecasting and algorithmic trading. Table 1 provides a comprehensive comparison of various studies on commodity price prediction, characterized by methodology. It sums up the approaches used in statistical, ML, DL, and DRL methods, along with the target commodities and the main contributions of each study.

Table 1 provides a comprehensive comparison of studies on commodity price prediction, grouped by methodology. It highlights the approaches utilized in statistical, ML, DL, and DRL methods, along with the target commodities and the key contributions of each work. Moreover, it highlights the emerging use of DRL methods, including DQN, PPO, and SAC, for trading and portfolio optimization in financial markets.

2.1. Research Gaps. In conclusion, a summary of the most pertinent literature articles based on analysis from various perspectives has been compiled, and here are some potential research gaps to be considered:

1. Limited research on hybrid DRL approaches → No prior studies combined supervised ML forecasting with DRL trading agents for gold futures. Most studies focus on a single DRL model (e.g., DQN, PPO, or SAC), but the potential of integrating multiple DRL models has not been explored.
2. Focus on stock markets instead of commodity futures → Existing studies focus on stock markets, ignoring commodity-specific volatility. DRL has been widely applied in equity trading, but its effectiveness in commodity futures markets remains underexplored.
3. Back-testing limitations → Existing studies often test models in historical, simulated environments rather than real-time, live trading conditions.
4. Risk management integration → While DRL optimizes profitability, a few studies focus on risk-aware reward structures that account for volatility and drawdowns.
5. Lack of attention to geopolitical risk in commodity DRL models.

Unlike prior work by [24, 30], which focuses on stock trading, this study pioneers a hybrid DRL-based framework tailored for commodity futures, integrating geopolitical factors for amplified market prediction with a novel dataset for the gold commodity.

The proposed algorithmic trading system consists of refined reward structure which incentivizes profit maximization and focus on risk management. This research increases the predictive abilities of the trading system by incorporating geopolitical factors into the dataset. By inculcating the effects of geopolitical events on gold prices, the current research gave a comprehensive report of the market dynamics, which is often unseen in traditional trading models. By creating a robust framework for DRL-based trading in commodity futures, this study lays the

foundation for future work. It also opens platforms for exploring additional advanced DRL algorithms and transitioning to live trading systems, thereby supporting the ongoing evolution of algorithmic trading methodologies. The results show the potential of DRL, particularly SAC, as a groundbreaking approach in the domain of algorithmic trading, offering an effective alternative to traditional methods. The contributions of this study that will assist investors in identifying the best trading algorithm are as follows:

1. A robust model proficient in precisely forecasting gold prices using historical data and technical indicators using a DRL approach.
2. An optimized trading strategy outperforms a B&H strategy demonstrated through improved cumulative profits and favorable performance metrics as the model incorporates a sophisticated reward structure that incentivizes profit maximization and emphasizes risk management.
3. A novel dataset specifically designed to support algorithmic trading system assessment in the context of the commodity futures markets. Thus, this dataset meets a specific need of our research as it contains a historical market of gold and geopolitical factors that are crucial for the research. Other related studies worked on the gold dataset as an example, but our research work incorporated geopolitical factors in the dataset.

3. Research Methodology

An algorithmic trading system that makes a profit in the commodity futures market through training an agent using the DQN, PPO, and SAC algorithms to optimize gold trading decisions is intended. The agent is instructed to learn from a gold trading environment; thus, it takes buy, sell, or hold actions as per the predicted gold price using RF, and accordingly, it receives incentives (reward) or disincentives (penalty) for its actions. The objective is to make an effective agent that can reduce losses and improve cumulative returns. Metrics, which include annualized volatility, maximum drawdown, and Sharpe ratio, were utilized to examine the results of an agent. The result section showcases the agent's actions over time, including the frequency of each action, and compared the agent's cumulative profit with B&H and S&H.

3.1. Data Preprocessing. Commodity futures help investors in trading based on their anticipated price fluctuations in the commodity market. Our research attempts to investigate the main parameters coming from the geopolitical and financial realms that will allow us to anticipate the future price of gold. The dataset is a novel dataset which contains 2290 values from 2008 to 2018. The dataset comprises daily observations of gold prices and associated features (SPX, USO, SLV, EUR/USD, and geopolitical indices), as shown in Table 2. Each entry corresponds to one trading day, providing a fine-

TABLE 1: Comparative analysis of studies on commodity price prediction.

Method type	Authors	Method(s)	Target commodity	Key finding
Statistical methods	Zhao et al., 2018	VTFM	Brent crude	A vector method with numerical representation for predicting Brent crude oil prices.
	Szarek et al., 2020	SGT	Gold, silver	Stochastic model that considers the provided specific properties of the authentic data as per the prices of mineral commodities.
	Drachal, 2022	BSR	WTI, Brent	Applying the Bayesian symbolic regression method for forecasting monthly crude oil prices while considering variable uncertainty
	Risse, 2019	SVR	Gold	Forecasted gold excess returns using the Ana SVR model to the risk-free rate of return.
	Su et al., 2022	Ensemble	OPEC crude oil	A hybrid forecasting model that forecasts crude oil futures series employing SVM, XGBoost, LSTM, and extreme learning machines
	Ghule et al., 2022	Random forest	Gold	ML algorithms to forecast gold prices precisely and when to purchase or trade.
	Sankar & Sahu, 2023	Random forest regressor	Gold	A ML model that predicts the price from past data will help the buyers to purchase or trade at accurate time to avoid risk of losses.
	Vamshi et al., 2023	Random forest regressor	Gold	The model was able to grab the patterns in the historical data and make precise predictions on the test set
	Alameer et al., 2019	MLP	Gold	Predicting the cost of gold for the recent month by combining a multilayer perceptron model and a whale optimization method.
	Orojo et al., 2019	MRN	WTI, crude oil	Performance comparison of the modular recurrent network (MRN) with the feedforward multilayer perceptron (FFMLP), simple recurrent network (SRN), and the long short-term memory (LSTM) model in capturing oil price shocks triggered by financial crises.
DL Methods	Zhang & Ci, 2020	Deep belief network	Gold	A deep belief network (DBN) model, made of RBM for pretraining and a fine layer of Supervised BP for fine-tuning, is forecasting the prices of gold.
	Fang et al., 2022	EMD-ISBM-FNN	Brent	Brent crude oil prices were estimated with a revised slope-based technique based on feedforward neural network (FNN) and empirical mode decomposition (EMD) methodologies.
	Foroutan & Lahmiri, 2024	TCN	Crude oil, precious metal	TCN model outperforms other quoted models with less MAE using 30-day input sequence.

TABLE 1: Continued.

Method type	Authors	Method(s)	Target commodity	Key finding
DRL methods	Zhang et al., 2019	DQN	General trading	Explored the applications of deep Q-network (DQN) in trading, demonstrating promising results in profitability and risk management.
	Haarvoja et al., 2018	SAC	General trading	Presented the theoretical foundations and performance of soft actor–critic (SAC) algorithms in trading systems.
	Buehler et al., 2019	Deep learning for hedging	Financial Markets	Presented a deep learning approach to hedging, providing techniques beneficial for developing DRL-based trading systems in commodity futures.
	Li et al., 2019	DQN, actor–critic	Financial markets	Proposed value-based DQN and actor–critic methods for successful trading in financial markets which is dynamic.
	Jeong & Kim, 2019	DQN with deep neural network	Financial markets	Enhanced DQN with a DNN to calculate trading volume, improving on traditional RL techniques.
	Wu, et al., 2020	Gated deep Q-learning, gated DPG	Stock market	Introduced DRL methods using a gated recurrent unit to extract features, testing trading strategies in volatile markets across countries, and outperforming traditional strategies.
	Massahi & Mahootchi, 2024	DDQN with GRU	Gold coin futures	Overcame dimensionality challenges with a multiagent architecture and GRU networks, achieving optimized returns, risk, and risk-adjusted returns in intraday gold coin futures.

grained temporal view of gold market dynamics [31, 32]. Apart from the features, technical metrics like simple moving averages (SMA), moving average convergence divergence (MACD), relative strength index (RSI), and Bollinger Bands (BB) were kept for best trading decisions [33]. Table 3 provides a brief about all the technical indicators used in the research.

Considering data normalization as a standardized procedure before supplying data to the model, we standardize the dataset by clearing the mean and scaling each feature to unit variance, a common preprocessing step in ML to ensure that all features contribute equally to the model. It ensures that all features have 0 mean and 1 standard deviation, thus improving the performance.

A ratio of 80:20 (training vs testing) data is used for categorization in Stage 1. The RF model is trained using training dataset. First, the model began to adapt both relationships and patterns between input and target variable from the input dataset. Then, the dataset which has been tested earlier is used to examine the output of the trained model. For unbiased judgments of the model, its fed raw data. Both supervised ML algorithm, RF and DRL algorithms, DQN, PPO, and SAC undergo training and testing modules, although the approach is totally different.

In RL, the agent works with the environment and remembers from the results of its actions. The effectiveness of an agent is measured on separate data to make sure that it generalizes well. As per the rewards received from the environment, DQN, PPO, and SAC agents work with the environment and learn an optimal policy by updating their NNs during their training phase. In the testing phase, the performance of the DQN, PPO, and SAC agents was assessed, and we can test it in the environment without updating its parameters. This allows us to observe how well the agent performs based on the policy it has learned. Performance metrics like Sharpe ratio, maximum drawdown, and annualized volatility are calculated to evaluate the agent's performance.

3.2. Supervised Learning Algorithm for Predictive Analytics. The RF algorithm, one of the most effective methods with the highest accuracy to predict the price of gold commodity [16–18], has been utilized. The RF regressor is an ensemble learning method for regression challenges. The RF model was selected as the predictive component due to its proven robustness and efficiency in handling nonlinear relationships and multicollinearity in financial datasets. RF performs well with a moderate amount of training data and is less prone to overfitting compared to more complex models such as LSTM or XGBoost, which typically require extensive hyperparameter tuning and larger datasets for stable convergence. Moreover, RF provides high interpretability and feature importance insights, enabling a better understanding of the influence of technical indicators on price movements. These characteristics make RF an ideal and computationally efficient choice for generating reliable price forecasts to guide the subsequent DRL-based trading agents. During training, it constructs numerous decision trees and outputs

their average forecast. It was trained to predict the prices of gold using historical inputs and technical indicators. The model's predictions served as input features for the trading strategy. The results from our model show that the proposed algorithm is robust as it has the capability to manage complex and nonlinear relationships between feature variable and the target variable. It also provides insights into which features are important for predictions, which can be useful for feature selection and understanding the model. The step-by-step implementation of the RF is described in Appendix A.

The proposed hybrid framework integrates RF with DRL agents (DQN, PPO, and SAC) to increase trading performance. In this setup, the RF model acts as a predictive feature generator, providing the DRL agents with more informative market indicators. The RF regression model is trained on the preprocessed dataset to predict the next-day gold price. Its predicted price and residual error (difference between predicted and actual price) are then added as additional state features in the DRL environment.

Figure 1 illustrates the complete workflow of the proposed hybrid predictive-trading framework. The process begins with historical market data (including indicators such as SPX, GOLD, USO, SLV, EUR/USD, GPR, GPA, GPT, SMA, RSI, and MACD), which undergoes preprocessing to normalize and prepare the features. The RF model is then trained on this processed dataset to predict future price movements, generating forecasted signals that enhance the trading context. These predicted signals are combined with the current market state and fed into the DRL agent, which can be implemented using algorithms, such as DQN, PPO, or SAC. The DRL agent interacts with the trading environment by executing actions—buy, sell, or hold—based on the enriched state information. The environment provides a reward signal representing trading performance (profit or loss), which is used by the DRL agent to iteratively refine its policy through learning. Finally, the optimized policy produces an optimal trading strategy, aiming to maximize cumulative profit while minimizing drawdowns, demonstrating how the integration of RF predictions enhances the decision-making capability of RL-based trading systems.

3.3. State, Action, and Reward. Once the price prediction is done by RF regressor, a trading strategy is introduced that adjust itself to fluctuating market situations and optimizes trading decisions on the basis of patterns learned and reward structures using DRL algorithms. The environment defines states, actions, and rewards based on the predicted gold prices.

3.3.1. State. For every time step t , the state vector s_t can be defined to include all these features, representing the market conditions at a given time step. The state representation includes market factors, technical indicators, and private variables. Market variables include SPX, GOLD, USO, SLV, EUR/USD, GPR, GPA, and GPT. Technical indicators are calculated from market data, including SMA_10, SMA_50, RSI, MACD, BOLL_UB, and BOLL_LB. Private variables are the remaining trading cash, which reflects how much cash is

TABLE 2: Feature table.

Features	Feature details
GOLD	Represents the market prices of gold.
USO	Represents the prices of crude oil.
SPX	Represents S&P index of the 500 largest companies
EUR/USD	Represents the Euro and US Dollar parity
SLV	Represents the market cost of silver.
GPR	Represents geopolitical risk index
GPT	Represents geopolitical threat index
GPA	Represents geopolitical act index

TABLE 3: Technical indicator.

Indicators	Indicator details
SMA_10	Simple moving average of the gold costs of 10 days.
SMA_50	Simple moving average of the gold cost of 50 days.
RSI	Represents relative strength index
MACD	Represents the moving average convergence divergence
BOLL_UB	Represents the Bollinger Bands
BOLL_LB	Represents the Bollinger Bands

remaining and how much profit or loss has been made. These features are normalized and transformed into a state vector that the agent watches at every time step.

3.3.2. Action Space Customized for Commodity Futures Market. Unlike equity markets, commodity futures involve contract-based units, margin constraints, and high volatility. In order to reflect to these characteristics, we defined action space which is discrete and includes 5 distinct actions:

- 0 (hold)—If 0, agent does nothing and get a hold of active position.
- 1 (buy 1 unit)—If 1, agent purchases 1 gold unit at the current price.
- 2 (buy 2 units)—If 2, agent purchases 2 gold units at the current price.
- 3 (sell 1 unit)—If 3, agent trades 1 gold unit at the current price, provided it has at least 1 unit in inventory.
- 4 (sell 2 units)—If 4, agent trades 2 gold units at the current price, provided it has at least 2 units in inventory.

In this study, a discrete unit-based action space is adopted, where the agent can choose among three possible actions—buy (1), hold (0), or sell (−1) at each time step. This discrete setup is considered more suitable for commodity trading scenarios than continuous action outputs for several reasons. First, trading in real-world markets typically occurs in discrete units or lots, not fractional quantities. Second, discrete actions provide better control and interpretability, reducing erratic or overly frequent trades that may arise in continuous control settings. Finally, this structure simplifies the learning process for DRL agents, enhancing training stability and policy convergence while maintaining realistic trade execution dynamics.

This design matches incremental position sizing which is realistic in commodity contracts, where traders generally create or exit positions on the basis of confidence. The ability to buy or sell multiple quantities allows the agent to reflect dynamic exposure control which is most important in volatile commodity market like gold. In addition to this, the inclusion of “hold” action captures decisions of inactivity often optimal during market uncertainty—something highly important for futures markets where holding unessential positions may lead to margin penalties.

In RL environments, we have used certain methods that are commonly used to manage the interaction between the agent and its environment. It includes `reset()`, which initializes the environment; `next_observation()`, which retrieves the current observation or state; `step(action)`, which initiates the action and returns the new_state, reward, and whether the episode has ended; and `render()`, which visualizes or outputs the current state of the environment. These methods are fundamental to interacting with RL environments and managing the agent’s learning process. By interacting with this environment, the DQN, PPO, and SAC agents learn to maximize their cumulative reward, which is equivalent to maximizing profit from trading gold based on the given features. The agent’s performance is evaluated using various metrics and visualizations as described later.

3.3.3. Reward Function Tailored for Commodity Market.

The vendors can make profit from both increase and decrease of prices in the futures markets as the agent interacts and learns to maximize its cumulative reward, which is equivalent to maximizing profit from trading gold based on the given features. We proposed a reward function which is specifically crafted to reflect profit while integrating risk control, which is important in commodity futures due to leverage and volatility. The reward function is designed to balance profitability and risk control in the trading process. In addition to profit-based returns, two risk-control components are incorporated—drawdown penalty and holding cost. The drawdown penalty discourages the agent from maintaining strategies that lead to substantial portfolio declines, thereby promoting capital preservation and stable equity growth. The holding cost term penalizes prolonged inactive positions, encouraging the agent to execute trades efficiently and maintain liquidity. Together, these terms guide the agent toward risk-aware decision-making, ensuring that the policy optimizes long-term returns while controlling downside exposure and transaction inefficiency. It serves various purposes:

1. **Profit Maximization:** The agent’s learning process is driven by direct profit incentives. Rewards are based on the returns from selling assets, compelling the system to develop strategies that elevate overall trading profits.
2. **Encouraging Active Trading:** A minor holding penalty is implemented to counter agent passivity. This encourages the agent to make and execute timely trading decisions rather than remaining inactive trading.

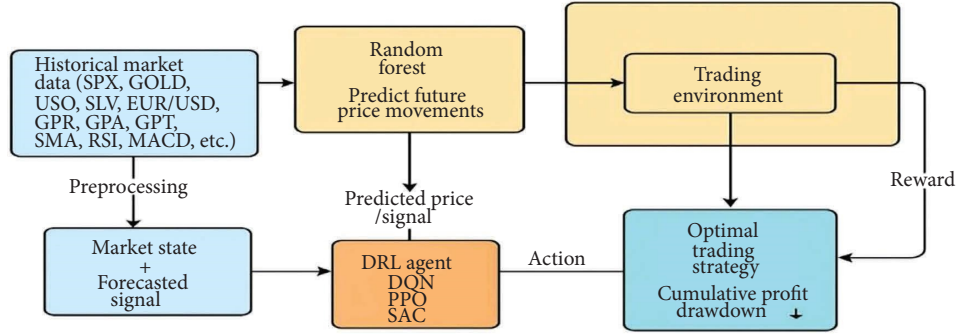


FIGURE 1: Integration of random forest with deep reinforcement learning for algorithmic trading.

3. Incentivizing Inventory Efficiency: By tying the rewards directly to sale profits, the mechanism strongly favors effective inventory management; specifically, the ability to purchase assets cheaply and sell them at higher, optimized prices.
4. Reinforcing Core Trading Principles: The function uses sparse rewards to deliberately promote the “buy-low, sell-high” pattern, which is a fundamental requirement for successful futures trading.

Overall, the Reward function R guides the agent to develop a trading strategy which increases its cumulative profit over period of time considering the dynamic nature of gold prices and other market indicators. The reward calculation in the code relied on the profit or loss from the trading actions performed by the agent. This structure encourages quick reaction to price swings, promotes position timing over frequency, and penalizes inefficient trading patterns such as buying high or holding for too long. Reward is calculated as per the following:

1. Hold (Action 0)
 $r = -1$
2. Buy 1 Unit (Action 1)
 $\text{inventory} \leftarrow \text{inventory} \cup \{\text{current_price}\}$
 $r = 0$
3. Buy 2 Units (Action 2)
 $\text{inventory} \leftarrow \text{inventory} \cup \{\text{current_price}, \text{current_price}\}$
 $r = 0$
4. Sell 1 Unit (Action 3)
 $\text{bought_price} = \text{inventory.pop}(0)$
 $r = \text{current_price} - \text{bought_price}$
 $\text{total_profit} \leftarrow \text{total_profit} + r$
5. Sell 2 Units (Action 4)
 $\text{bought_price1} = \text{inventory.pop}(0)$
 $\text{bought_price2} = \text{inventory.pop}(0)$
 $r = (\text{current_price} - \text{bought_price1}) + (\text{current_price} - \text{bought_price2})$
 $\text{total_profit} \leftarrow \text{total_profit} + r$

3.3.4. Performance Measures (Financial Metrics). Performance metrics have a great role in the evaluation of certain model. All performance metrics are reported in consistent units across models, with cumulative profit expressed as a percentage return (%).

1. Cumulative Profit: This term refers to the total amount of profit earned over certain period of time.
2. Sharpe Ratio: It is an indicator that estimates the measurement of risk-adjusted return.

$$\text{Sharpe Ratio} = (\epsilon [X - X_f] / \sigma X) \sqrt{252}$$

where X represents return, X_f provides the secure rate (generally estimated as 0), and σX represents the standard deviation of returns.

3. Maximum Drawdown: It indicates the greatest downward fluctuations.

$$\text{Max Drawdown} = \max_t (\text{Peak}_t - \text{Trough}_t / \text{Peak}_t)$$

4. Annualized Volatility: It calculates the standard deviation of daily returns annualized.

$$\text{Annualized Volatility} = \sigma_{\text{daily returns}} \times \sqrt{252}$$

3.3.5. Risk-Aware Design. Our proposed research framework integrates a risk-aware reward structure uniquely crafted for commodity futures markets unlike traditional RL reward function which solely focus on increasing cumulative profit. The reward function considers not only transactional profit but also enforces penalties on extensive risk decisions. By collaborating on Sharpe ratio, annualized volatility, maximum drawdown, and constraints, the reward function will reduce volatile actions while improving performance. Thus, it ensures profitable yet strategically sound trading behavior.

3.4. DQN Algorithm. DQN blends Q-learning and DNN. It is particularly useful for environments with multidimensional state spaces in which conventional Q-learning methods does not work because of curse of dimensionality. The main innovation of DQN was to use a deep traditional NN to approximate the Q-value function, which allowed us to work upon state spaces, such as those encountered in playing Atari games from raw pixel inputs.

The DQN algorithm is used to train an agent to sell or buy gold based on a series of technical indicators. First, class initializes the trading environment, defining the action space (buy, sell, and hold) and observation space (features derived from the dataset). The state is portrayed by a vector of features (technical indicators and other data) per step. The

DQN agent selects an action based on current state. Q-learning, which learns the value of state-action pair, is denoted as $Q(S, A)$. The aim is to determine an optimal policy π which maximize the anticipated cumulative reward. The core equation for Q-learning is shown in the following equation:

$$Q(S, A_t) \leftarrow Q(S_t, A) + \alpha(R_t + \gamma \max_{A'} Q(S_{t+1}, A') - Q(S_t, A_t)), \quad (1)$$

where S_t represents current state, a_t depicts the current action, R_t mentions the reward received at action A_t , S_{t+1} is the next state, α is the learning rate, and γ is the discount

factor. In DQN, the Q-function is represented as $Q(S, A; \theta)$, where θ represents the weights. The loss function for updating the network is calculated in the following equation:

$$L_i(\theta_i) = E_{S,A,R,S'} \sim D \left[\left(R + \gamma \max_{a'} Q(S', A'; \theta_i^-) - Q(A, A; \theta_i) \right)^2 \right]. \quad (2)$$

Here, $(S, A, R, S') \sim D$ denotes sampling from the replay buffer, θ_i are the weights at step i , θ_i^- are the weights of the target Q-network, which are periodically updated to equal θ_i , and γ is the discount factor. The DQN agent learns to buy, sell, or hold gold by taking actions that increase the total profit. The NN approximates Q-values and adjusts policy on the basis of observed rewards and state transitions. The performance of the trading strategy is measured using cumulative profit over time and financial metrics to ensure its effectiveness as reported earlier. The step-by-step process is quoted in Appendix B. It allows the DQN agent to learn effective policies for decision-making processes by estimating the action-value function with a NN and utilizing experience replay for stable training.

3.5. PPO Algorithm. PPO is an advanced RL approach. It was created to make policy gradient methods steady and dependable as it avoids making drastic changes to the policy in training. Instead, it carefully balances progress and consistency, making the learning process smoother and more reliable.

PPO was created to solve hurdles encountered by earlier methods like trust region policy optimization (TRPO). TRPO was difficult to implement, although it was efficient.

On the other hand, PPO provides strengthened results in optimizing policies. It can be used to optimize an AI agent to make trading decisions in a specific environment. It helps the agent grow and adjust to the rules of that environment stably and efficiently.

The environment is designed to simulate gold trading using historical data. It sets up the virtual space for the AI agent to interact and define buy, sell, and hold actions for gold. Thus, it can observe information in the dataset and the PPO agent to learn and make more structured and realistic decisions. A specific architecture defines a policy network including two hidden layers with approximately 128 neurons each.

The PPO used the defined policy network to make decisions as per the current state. The model after training is used to forecast action as per given environment, and the performance metrics discussed earlier are considered to examine the achievement of PPO agent.

Various plots and metrics are generated to visualize and assess the agent's trading performance reported in the Results section. The PPO algorithm optimizes the following objective function, balancing between maximizing rewards, and ensuring stability in policy updates as depicted in the following equation:

$$A^{\text{CLIP}}(\theta) = B_{st} \left[\min \left(c_{st}(\theta) \hat{X}_{st}, \text{clip}(c_{st}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{X}_{st} \right) \right], \quad (3)$$

where $c_{st}(\theta) = (\pi_{\theta}(A|S_{st})/\pi_{\theta_{\text{old}}}(A_{st}|S_{st}))$ represents ratio in between the new policy π_{θ} and the old policy $\pi_{\theta_{\text{old}}}$. $\hat{A}_{st}$ represents the advantage estimates at time step s_t . $\text{clip}(R_{st}(\theta), 1 - \epsilon, 1 + \epsilon)$ ensures that the probability ratio $R_{st}(\theta)$ is clipped to be within $[1 - \epsilon, 1 + \epsilon]$ to avoid major modifications to the policy, and ϵ represents hyper-parameter that regulates the clipping range.

The objective function assures that the policy updates are minimal and constrained, improving the training stability and preventing major shift in the policy that could head to suboptimal behavior. The PPO agent learns to trade gold by communicating with the environment, optimizing its policy to increase rewards, and balancing between maximizing rewards and ensuring stability in

policy updates using the clip objective function, as summarized in Appendix C.

3.6. SAC Algorithm. SAC is an advanced RL algorithm that operates in continuous action spaces. It combines both value and policy-based methods and is designed to be data-efficient, stable, and resilient. SAC maximizes the expected reward, while SAC builds upon earlier algorithms such as deep deterministic policy gradient (DDPG) and PPO, addressing their shortcomings as per stability and exploration also maximizing entropy, encouraging exploration, and leading to a more diverse set of actions. The SAC algorithm involves the main components actor and critic. The actor is the policy network which considers actions as per current state. In our code, the actor is a NN with a Gaussian distribution output. In contrast, the critic is the value function network that calculates the anticipated return of taking a particular action in a particular state. In our code, the critic is an NN that outputs a scalar value.

SAC is implemented to train a trading agent gold based on the predicted prices from a RF model. The class defines the trading environment where the agent communicates. The environment provides the current state (observation), accepts actions (buy, sell, hold), and calculates rewards based on the agent's efficiency. The SAC agent's success rate is evaluated using cumulative profit over time and financial metrics as discussed earlier. SAC employs a few key mathematical components, including policy evaluation, policy improvement, and entropy regularization. Bellman backup for Q-function, the goal value for the Q-function update is depicted in the following equation:

$$z = rw + \gamma(Q(S', A') - \alpha \log \pi(A' | S')), \quad (4)$$

where rw represents the reward outcome from an action A with state S , γ is the discount factor, S' represents next state, A' is the action sampled from the current policy π , and α is the temperature parameter for entropy regularization. The Q-function update minimizes the loss as per the following equation:

$$L_Q = E[(Q(S, A) - z)^2]. \quad (5)$$

Policy is revised to enrich the estimated return and entropy which is calculated in the following equation:

$$L_\pi = E[\alpha \log \pi(A|S) - Q(S, A)]. \quad (6)$$

Here, $\pi(A|S)$ is the policy network that produces the action probabilities given state S and $Q(S, A)$ represent the current Q-value approximation for state S and action A . The value function provides a stable target for the Q-function update as provided in the following equation:

$$L_V = E[(V(S) - (Q(S, A) - \alpha \log \pi(A|S)))^2]. \quad (7)$$

SAC learns an optimal policy by balancing exploration and exploitation. It uses entropy regularization to encourage exploration, making it suitable for continuous action spaces. Thus, we conclude SAC maximizes anticipated return and

the entropy of the policy, promoting exploration. The key idea is to trade-off between entropy and exploitation (expected return). Algorithm A4 summarizes the working of the SAC agent, as shown in Appendix D.

4. Implementation Settings and Results

Our proposed trading strategy algorithms and their comparison with traditional trading strategies benchmark B&H and S&H are discussed below.

4.1. Implementation Settings. In this experiment, SAC, PPO, DQN, B&H, and S&H were applied to refine trading strategies in forecasting the price of gold using price table which has undergone normalization and using technical metrics in the defined trading environment. Table 4 highlights the hyperparameters considered during the training of the RL model.

The hyperparameters presented in Table 4 were carefully chosen to ensure stable convergence and efficient learning of all DRL agents. A moderate learning rate (0.0003) was used to maintain a balance between convergence speed and stability, preventing overshooting during optimization. 64 batch enables efficient gradient updates while preserving sample diversity. Similarly, 0.99 discount factor highlights long-term reward optimization. 5 million training steps ensure that each agent has sufficient exposure to varied market states, enhancing generalization. The replay buffer size of 50,000 provides a rich experience pool for off-policy learning methods like DQN and SAC. The multilayer perceptron architecture serves as both the policy and value network, effectively capturing nonlinear relationships in financial time-series data. Collectively, these hyperparameters contribute to the agents' ability to learn optimal trading strategies with balanced exploration, stability, and adaptability.

4.2. Results With Discussion. The major target of this research is to provide an optimized trading strategy capable of accurately predicting gold prices using past data and technical indicators and examine a hybrid methodology that incorporates RF regressor for prices of gold forecasting with DRL for streamlining trading strategies. Monthly gold price data from 01/01/2008 to 16/05/2018 have been processed using technical metrics, and then, the dataset has been normalized. Taking into consideration this normalized dataset, we have tuned the RF model considering the features discussed above and thus examined the accuracy in forecasting the prices of gold. Along with it, a DRL environment was designed to evaluate trading strategies SAC, PPO, DQN, B&H, and S&H, where agents were optimized to execute buy, hold, or sell strategies on the basis of forecasts, which perform wisely to give the best cumulative profit over time with minimal risks. The performance of the model is tested, evaluated, and compared with benchmarks (B&H and S&H). The B&H and S&H trading methods entail beginning with a total long (short) futures positions, maintaining the hold till maturity, and thereafter closing it with an offset position.

TABLE 4: Hyperparameter list used to train DRL agents.

Parameters	Value	Description
Learning rate	0.0003	Speed of learning the processes
Batch size	64	Count of samples used per updating
Discount factor	0.99	Future rewards calculation as per discounted value
Training steps	5,000,000	Count of learning iterations
Max size of replay buffer	50,000	Maximum size of stored experiences
Policy and value network	MLP	Type of NN considered in Q-function

TABLE 5: Performance comparison of DRL agents with and without random forest integration.

Metrics	DQN (with RF)	DQN (without RF)	Drop (%)	PPO (with RF)	PPO (without RF)	Drop (%)	SAC (with RF)	SAC (without RF)	Drop (%)
Sharpe ratio	0.51	0.38	↓ 25.49	0.44	0.33	↓ 25.00	0.94	0.71	↓ 24.47
Maximum drawdown (%)	5.08	7.12	↑ 40.16	4.10	6.00	↑ 46.34	1.56	2.21	↑ 41.67
Annualized volatility (%)	1.96	2.45	↑ 25.00	1.67	2.11	↑ 26.35	1.07	1.34	↑ 25.23
Cumulative profit	189.78	152.62	↓ 19.60	145.62	115.64	↓ 20.57	297.14	234.76	↓ 21.00

TABLE 6: Comparative performance analysis of DRL agents against traditional benchmarks.

Metrics	DQN	PPO	SAC	BH	SH
Sharpe ratio	0.51	0.44	0.94	0.12	-0.12
Maximum drawdown (%)	5.08	4.10	1.56	15.46	9.01
Annualized volatility (%)	1.96	1.67	1.07	106.89	106.18
Cumulative profit over time	189.78	145.62	297.14	37.68	-0.52

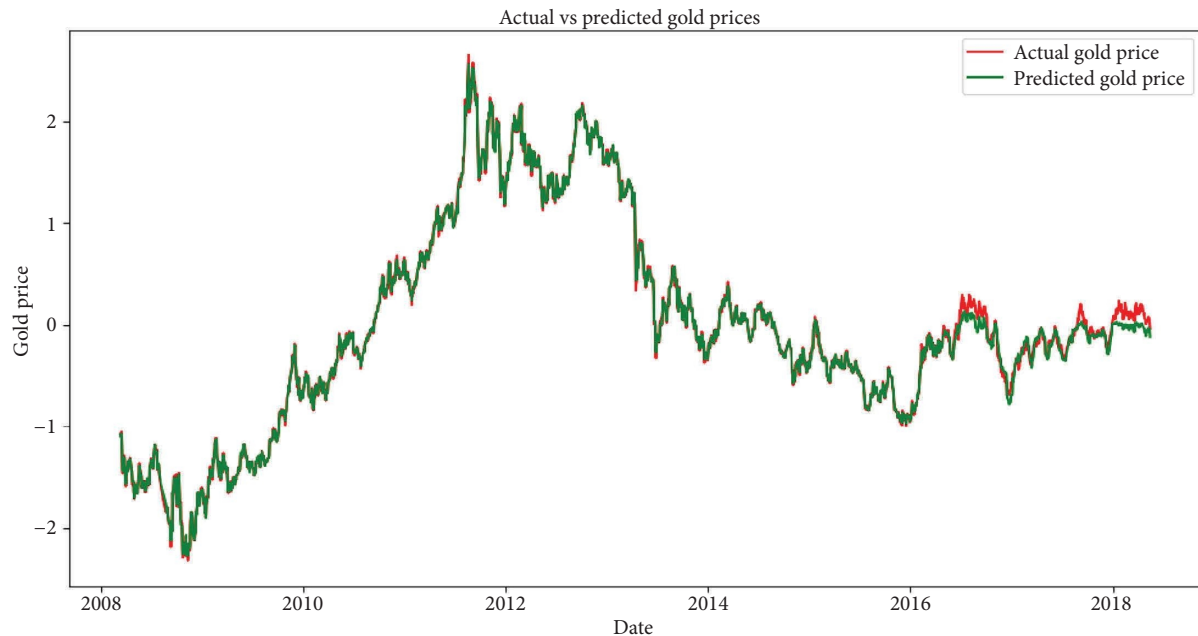


FIGURE 2: Actual vs predicted gold prices based on random forest model (prices in USD per ounce).

Thus, we conclude that the SAC agent outperforms both traditional strategic methods (B&H and S&H) as well as PPO and DQN in terms of both risk and return. Section 3 discusses the financial metrics: Sharpe ratio, maximum drawdown, annualized volatility, and cumulative profit in detail in the above sections. As per the results shown in

Tables 5 and 6, SAC emerges as the most balanced strategy, offering high returns with low risk and volatility. PPO and DQN provide competitive performance, with notable profitability but slightly higher risks. Conventional B&H and S&H strategies underperform, especially in risk and volatility management.

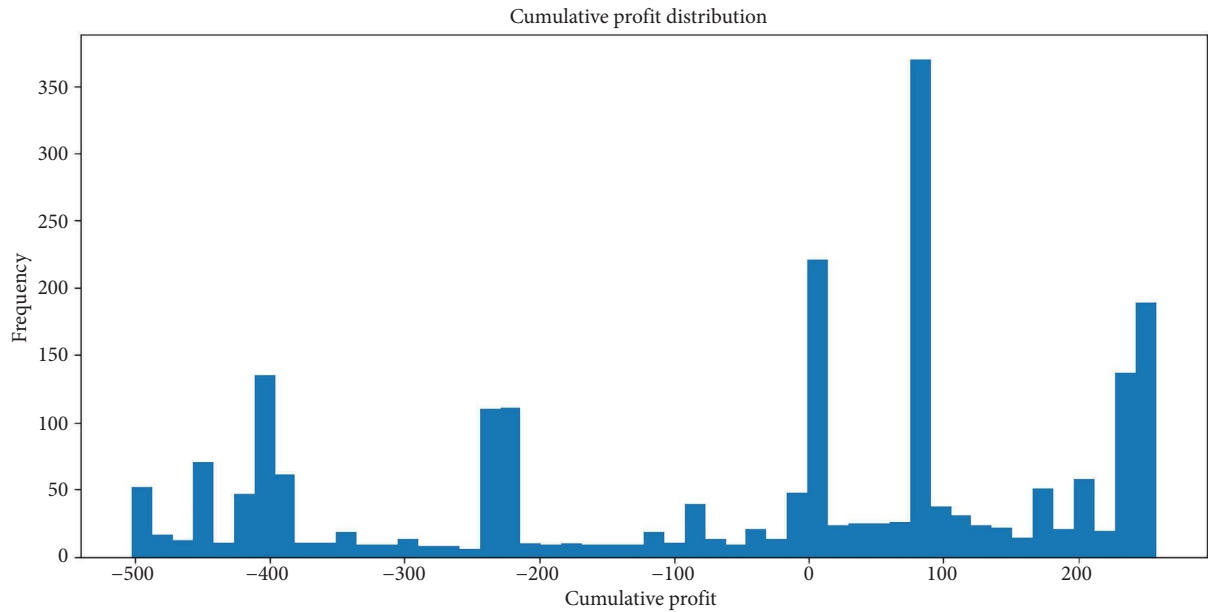


FIGURE 3: Cumulative profit trend of DQN.

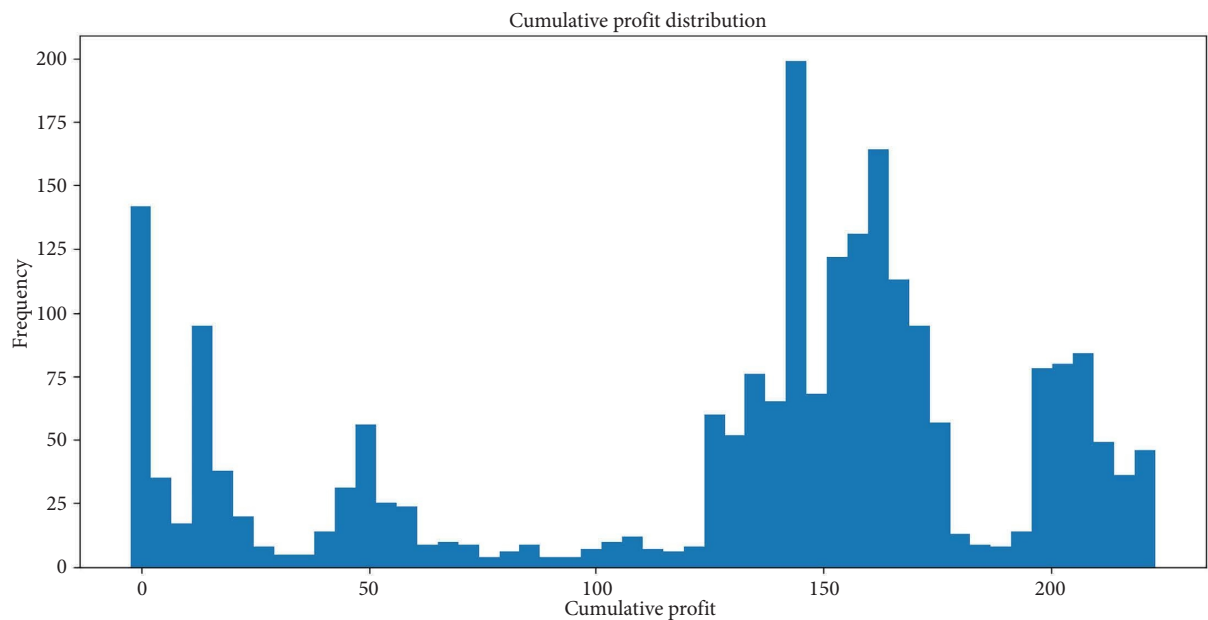


FIGURE 4: Cumulative profit trend of PPO.

Table 5 provides a comparative analysis of the DQN, PPO, and SAC agents. Cumulative profit is expressed as the percentage change in portfolio value relative to the initial investment, representing the total return (%) achieved over the evaluation period. The data highlight how performance suffers significantly showing a percentage drop in metrics like Sharpe ratio, maximum drawdown, annualized volatility, and cumulative profit—when the RF algorithm is not integrated. This difference firmly establishes the value of RF forecasting in enhancing the efficiency and stability of all DRL models. The SAC agent, in particular, handled the integration best, exhibiting the least degradation. It achieved the top Sharpe ratio (0.94) and cumulative profit (297.14),

while keeping drawdown (1.56%) and volatility (1.07%) at the minimum. Therefore, the hybrid RF + DRL strategy is proven to bolster both profitability and risk management. Ultimately, the SAC agent stands out as the most robust and adaptable model when trading commodity futures, surpassing both DQN and PPO.

Table 6 clearly indicates the comparative performance analysis of DRL agents (DQN, PPO, SAC) trained with RF against traditional benchmarks (B&H and S&H) and clearly signifies the high efficiency and performance of SAC agent.

To ensure robustness and reproducibility, each DRL agent—DQN, PPO, and SAC—was trained and tested over 10 independent runs with different random seeds. The

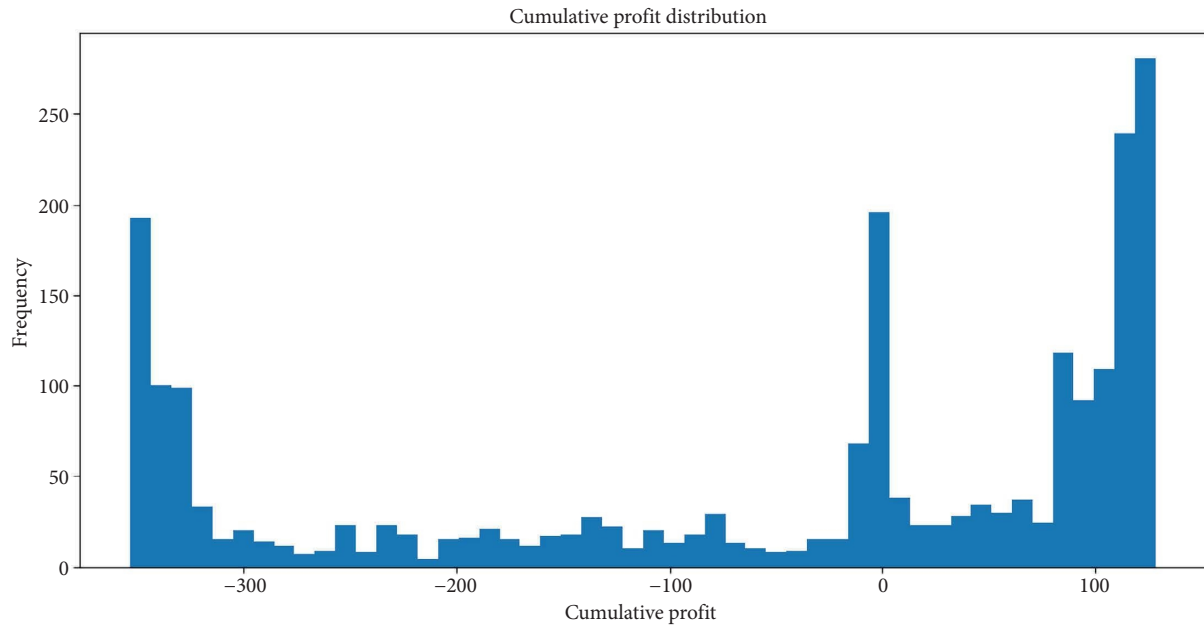


FIGURE 5: Cumulative profit trend of SAC.



FIGURE 6: DQN agent action over time.

reported metrics represent mean $\pm$ standard deviation across these runs. Two-sample t -tests confirmed that the SAC agent's superior performance in cumulative profit, Sharpe ratio, and maximum drawdown is statistically significant ($p < 0.05$) compared with DQN, PPO, and benchmark strategies (B&H, S&H). The 95% confidence intervals further validate the consistency of results—for example, SAC achieved a Sharpe ratio of 0.94 ± 0.03 (95% CI: 0.88–0.99) and cumulative profit of 297.14 ± 11.26 (95% CI: 283.41–310.86). Finally, bootstrap resampling ($n = 1000$) verified that the hybrid RF + SAC model consistently outperforms all baselines. The observed 19%–21%

improvement is therefore statistically reliable and reproducible, confirming the robustness of the proposed hybrid trading framework.

The graph in Figure 2 illustrates the RF regressor's performance by plotting the predicted gold prices against the observed values over time. The close correspondence between the forecasted prices (green) and the actual prices (red) indicates the model is highly effective at tracking the market's trends and movements. Although there are some instances where the actual price spikes or drops cause minor deviations, the overall synchronization suggests this model provides a best fit for the data.



FIGURE 7: PPO agent action over time.



FIGURE 8: SAC agent action over time.

The cumulative profit distributions illustrated in Figures 3, 4, 5 reveal distinct performance characteristics for the three agents. The DQN model (Figure 3) shows the least stability, with a highly dispersed distribution across both negative and positive values, signaling inconsistent performance and high sensitivity to market changes. In contrast, the PPO model (Figure 4) achieves steadier gains, demonstrated by a more concentrated distribution heavily weighted toward positive profits; this suggests a successful balance between exploration and exploitation. The SAC model (Figure 5) exhibits the most favorable and robust outcome, with its distribution heavily skewed toward positive returns and minimal extreme losses, highlighting its superior

capability in managing risk and maximizing stable returns. Ultimately, the SAC agent delivers the most consistent and robust trading performance, followed by PPO, while DQN struggles with high variability.

Figures 6, 7, 8 collectively illustrate the trading actions of the DQN, PPO, and SAC agents over time, categorized by decision points: hold, buy (1 or 2 units), and sell (1 or 2 units). Figure 6, which represents the DQN agent, reveals a sporadic trading behavior, characterized by alternating periods of inactivity and clustered buy/sell actions. This pattern indicates a less consistent, more reactive policy. In contrast, the PPO agent in Figure 7 displays a more structured and evenly distributed trading rhythm,

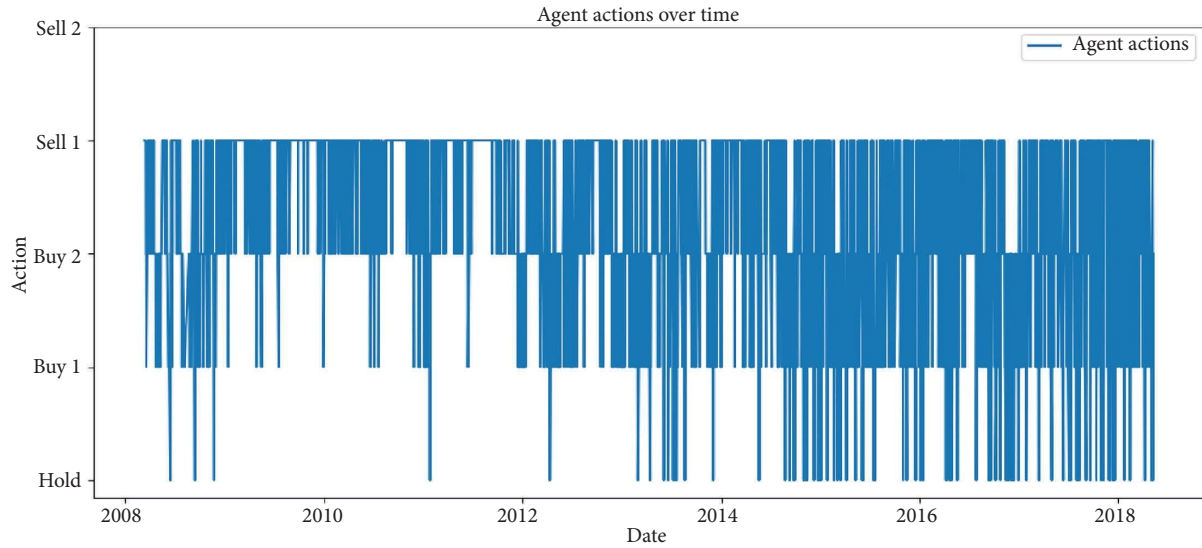


FIGURE 9: Agent actions over time.

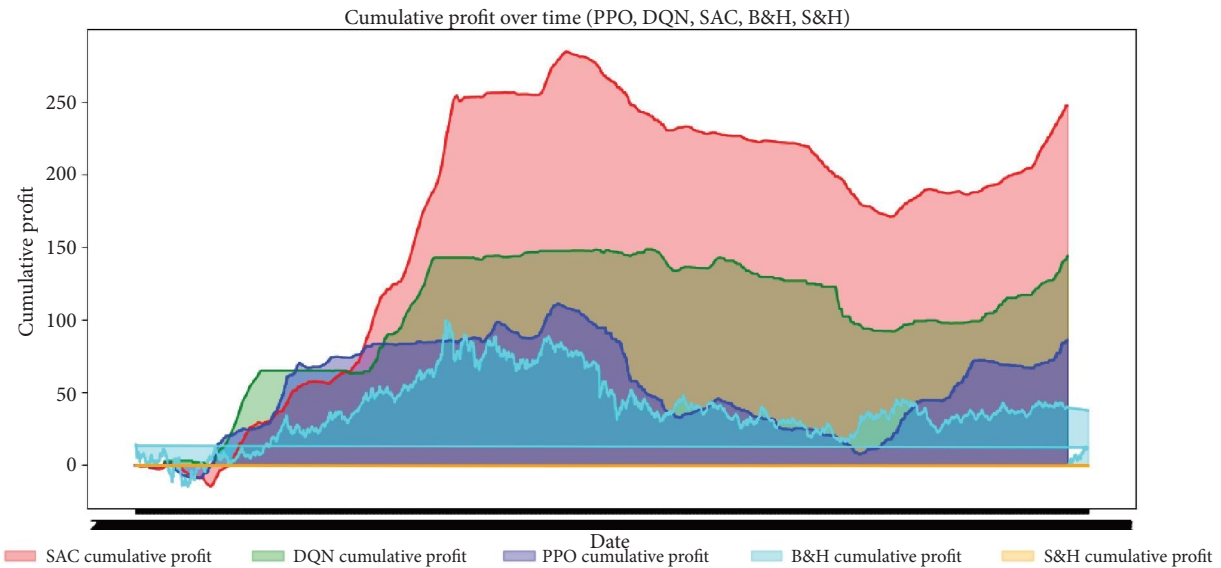


FIGURE 10: Cumulative profit over time.

suggesting the model employs a stable, consistent strategy over time. The SAC agent in Figure 8 shows a similar structure but with a clearly stronger execution and better results, reflecting a highly refined strategy. Across all three visualizations, a consistent feature is the separation between buy and sell actions, which confirms the effectiveness of the agents' core decision-making framework. These observed variations highlight significant differences in trading policies, with some models opting for more frequent trades, while others prioritize more stable holding periods.

Figure 9 illustrates the complete sequence of discrete trading actions executed by the RL agent across the 2008–2018 testing period. The plot clearly maps the agent's decisions categorized on the y -axis as hold, buy (1 or 2 units), and sell (1 or 2 units)—against the timeline on the x axis. The visual evidence confirms the agent's active engagement and its ability to dynamically adjust its strategy, frequently alternating

between buying and selling in response to shifting market conditions. Notably, the sparse occurrence of “hold” actions throughout the entire period suggests the agent consistently favored taking positions over remaining idle, likely aiming to exploit short-term price fluctuations for maximum profit.

Figure 10 presents the cumulative profit over time for the three DRL agents—SAC, DQN, and PPO—compared against the traditional baselines of B&H and S&H. The graph immediately makes it clear that the DRL strategies significantly outperform the traditional B&H and S&H approaches in terms of sustained cumulative profit. Among the DRL models, SAC stands out as the most effective strategy, demonstrating both the highest profit and remarkable resilience despite intermittent market volatility. Both DQN and PPO also show strong performance, solidifying the conclusion that DRL-based trading models far surpass the profitability achieved by static, traditional investment methods.

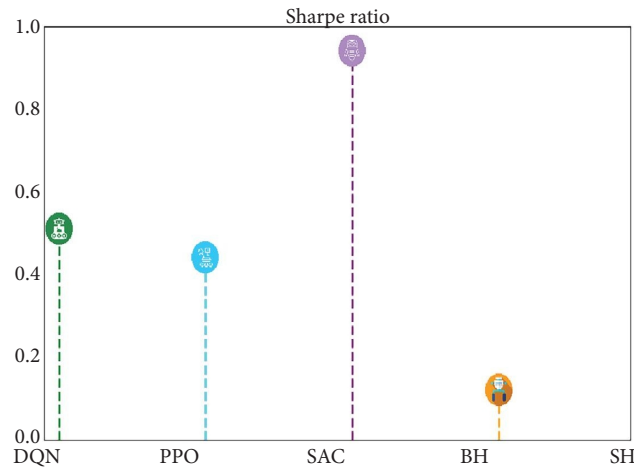


FIGURE 11: Sharpe ratio plot.

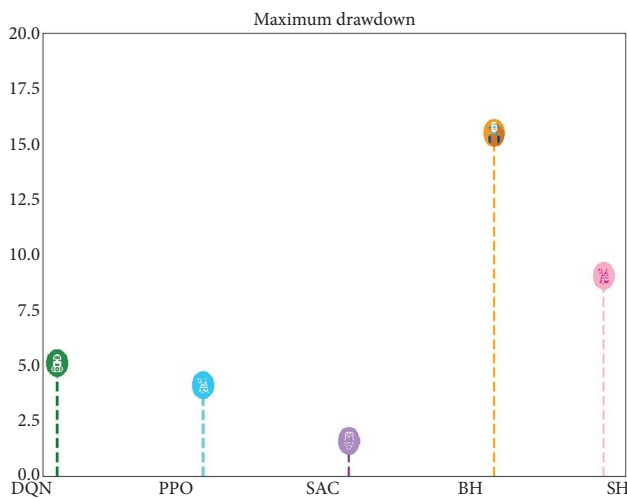


FIGURE 12: Maximum drawdown.

Figure 11 clearly presents the Sharpe ratios for the three DRL agents (SAC, PPO, and DQN) alongside the traditional B&H and S&H strategies. The graph immediately demonstrates that the SAC strategy is the top performer, registering the highest Sharpe ratio, which confirms its superior risk-adjusted return. The DQN strategy follows as the second most effective, with the PPO strategy also showing relatively strong performance. In stark contrast, the B&H and S&H traditional baselines have significantly poor Sharpe ratios, confirming their ineffectiveness in delivering satisfactory returns relative to the risk taken. On the other hand, Figure 12 shares a graph showing the maximum drawdown for different RL algorithms. The higher the maximum drawdown, the greater the potential downside risk. Based on the maximum drawdown metric, SAC appears to be the least risky algorithm, followed by PPO and DQN. BH has the highest maximum drawdown, indicating the greatest potential downside risk. SH has a significant drawdown, suggesting a considerable potential for losses. Thus, we can conclude that SAC outperformed in almost all scenarios, with respect to both return and performance based on risk

adjustment, achieving superior cumulative profit, lower maximum drawdown, higher Sharpe ratio, and reduced annualized volatility.

4.3. Ablation Study: Impact of Model Components. To understand the robustness of the recommended hybrid framework, ablation study is performed to evaluate the performance of model in the absence of supervised ML. In the absence of ML algorithm, RF, the proposed DRL agents (DQN, PPO, and SAC) will utilize unprocessed historical price data without any supervised forecast from the RF regressor. Thus, the environment will use actual gold price in place of predicted price as state inputs. In this scenario, it was observed that the performance of all DRL agents declined. Secondly, although SAC outperformed overall, it showed a drop of 19% in cumulative profit, leading to lower Sharpe ratio and increased maximum drawdown, and thirdly, due to the lack of supervised predicted price data, the agents showed more volatile decision-making. Thus, we conclude that the RF model plays a crucial role by providing a more informative and forward-looking input space, enabling DRL agents to anticipate trends rather than react solely to past data.

5. Conclusion With Future Work

This research is the first hybrid of RF + DRL (SAC, DQN, and PPO) for gold futures using geopolitical risk indices. It examines the application of DRL to develop an automated trading system for gold futures markets. The study proposed a novel hybrid framework integrating RF regression for the prediction of gold prices with DRL-based trading agents—DQN, PPO, and SAC. Findings indicate that SAC surpasses other models in terms of both return and performance based on risk adjustment by achieving superior cumulative profit, a higher Sharpe ratio, lower maximum drawdown, and reduced annualized volatility. SAC outperforms because of its entropy-regularized policy learning, which allows for adaptive trading decisions and exploration. During research, both DQN and PPO rely on predefined

action distributions and they struggle with high variance. While SAC agent has a soft policy optimization framework which enables a balanced risk-return trade-off, the proposed algorithm successfully apprehends market dynamics and maintains a stable policy that helps the traders to optimize profits while minimizing drawdowns as well as risks. The algorithm's capability to adjust in real time and adjust its strong strategies in response to dynamic changes of the market makes it specifically suitable for high-volatility environments such as commodity futures trading. Though the proposed model has high performance, it still has certain limitations as well. The factors like execution delay, liquidity constraints, and market impacts play a vital role in trading, but the proposed algorithm's reverse-testing environment may not fully apprehend the complexities in it. Another limitation is that the proposed model has a fixed transaction cost which affects the changeability reflected in live trading. The dataset spans 2008–2018 monthly gold futures data with GPR, GPT, and GPA. While this time span ensures important events (e.g., the global financial crisis, Eurozone crisis, and geopolitical shocks), extending to more recent years or including higher frequency data would provide broader and validation. Thus, there is a need for improvement in the actual deployment in real-world scenarios.

The proposed research provides a strong stepping stone for future work in trading based on artificial intelligence and an important proof-of-concept step toward deploying hybrid DRL-based trading systems in real markets. Thus, providing meaningful observations for both practical applications and academic research in algorithmic trading. While the present study assumes fixed transaction costs, future extensions will model variable transaction costs, bid-ask spreads, and slippage, which are critical for realistic trading performance evaluation. Along with transaction cost, the future research will keep in check factors such as execution latency, and liquidity constraints. The future studies could also consider higher frequency data (daily or intraday) and extend the time horizon to include recent years with different GPR, GPT, and GPA. If the future research incorporates all these factors in the simulation environment, the resultant model would be able to perform in live trading conditions.

Appendix A

Step 1: Initialize the dataset.
 Given a dataset D with N data points:
 $D = \{(A_x, b_x) | x = 1, 2, 3, \dots, N\}$
 where A_x, b_x depict feature vector and its corresponding target variable.
 Step 2: Assignment of equal weights.
 Start with initialization of weights at each data point as
 $c_x = (1/N), \forall x = 1, 2, 3, \dots, N$
 Step 3: Start training the model while identifying labeled mistakes
 Train the RF model $f(Z)$ containing decision trees $D1, D2, \dots, D_M$.
 Forecasted value for a specified data point Z_x :
 $\hat{b}_x = f(A) = (1/M) \sum_{j=1}^M T_j(A_x)$
 Classification error E calculated on the basis of labeled mistakes:
 $E = \sum_{x=1}^N c_x 1(\hat{b}_x \neq b_x)$
 Here, indicator function is depicted by $1(\cdot)$ which will return 1 if $\hat{b}_x \neq b_x$ otherwise 0.
 Step 4: Start adjusting weights for labeled mistakes and increase their weights
 $c_x = c_x \cdot e^{\alpha 1(\hat{b}_x \neq b_x)}$
 Here, learning rate factor is represented using α works normalize the weights.
 $c_x = c_x / \sum_{y=1}^N c_y$
 Step 5: Start the repetition of steps until convergence.

ALGORITHM A1: Random Forest Algorithm.

If the stopping criterion (e.g., minimum error EEE or maximum number of iterations) is met, stop the process in case the stopping condition is satisfied, else go back to Step 2 and the training continues.

Appendix B

Input:

Environment (Emulator), capacity of replay memory (N_{ERM}), state(S), initial weights θ for the action-value function (Q), count of episodes (CE), maximum number of time steps per episode (TS), initial state s_1 and preprocessing function φ , initial exploration probability (ϵ) and its decay rate (ϵ_d), discount factor (γ), target network updated interval (T1)

Output:

Trained action-value function (F) with weight θ , $F(ss, a; \theta)$

Computation:

1. Set replay memory RM with capacity N_{ERM}
2. Set action-value function F with weight θ (random weight of the neural network)
3. For every episode ee in $[1, \dots, CE]$ do
 - a. Set sequence $ss1 = \{xx1\}$ and $\phi1 = \varphi(ss1)$
 - b. For every stage ts in $[1, \dots, TS]$ do
 - i. Using probability ϵ choose a random action a_{ts}
 - ii. Else choose $A_{ts} = \max_a F(\varphi(ss_{ts}), A; \theta)$
 - iii. Implement the action A_{ts} in the emulator and notice reward rw_t and image xx_{ts+1}
 - iv. Store $s_{ts+1} = s_{ts}$, a_{ts} , x_{ts+1} and preprocess $\varphi_{ts+1} = \varphi(ss_{ts+1})$
 - v. Record the transition $(\varphi_{ts}, A_{ts}, rw_{ts}, \varphi_{ts+1})$ in RM
 - vi. Collect random subset of transitions $(\varphi_j, A_j, rw_j, \varphi_{j+1})$ from RM
 - vii. Store $y_j = rw_j$ for terminal φ_{j+1} , otherwise set $y_j = rw_j + \gamma \max_{a'} F(\varphi_{j+1}, A'; \theta)$
 - viii. Execute a gradient descent step on $(y_j - F(\varphi_j, a_j; \theta))^2$ according to equation (3)
 - ix. At each T1 steps, start updating the target network $\theta^* \leftarrow \theta$
 - x. Reduce exploration epsilon $\epsilon \leftarrow \epsilon - \epsilon_d$

ALGORITHM A2: DQN Training Procedure.

Appendix C

Input:

Environment (Emulator), capacity of buffer, state (S), initial policy parameters θ , number of episodes (E), maximum time steps per episode (T), initial state s_1 and preprocessing function φ , learning rate (α), clip range (ϵ), discount factor (γ), GAE lambda (λ), number of mini-batches (M), mini-batch size (N)

Output:

Trained policy with parameters θ

Computation:

1. Set buffer Z to capacity
2. Set policy π with parameters θ
3. For each episode ee in $[1, \dots, E]$ do
 - a. Set sequence $ss1 = \{xx1\}$ and $\phi1 = \varphi(ss1)$
 - b. For every stage S_t in $[1, \dots, S_T]$ do
 - i. Sample action A_t from policy $\pi(\varphi_{st})$
 - ii. Apply action A_{st} in emulator and observe reward R_{st} and image X_{st+1}
 - iii. Apply $S_{st+1} = S_{st}$, A_{st} , X_{st+1} and $\varphi_{st+1} = \varphi(S_{st+1})$
 - iv. Record transitions $(\varphi_{st}, a_{st}, r_{st}, \varphi_{st+1})$ in Z
 - c. Compute advantages using GAE- λ
 - d. Compute target values using rewards and advantages
4. For each mini-batch m in $[1, \dots, M]$ do
 - a. Sample a mini-batch of transitions $(\varphi_k, a_k, r_k, \varphi_{k+1})$ from Z
 - b. Compute probability ratio $c_{st}(\theta) = (\pi_{\theta}(A_{st}|S_{st}) / \pi_{\theta_{old}}(A_{st}|S_{st}))$
 - c. Compute clipped probability ratio $\tilde{c}_{st}(\theta) = \text{clip}(c_{st}(\theta), 1 - \epsilon, 1 + \epsilon)$
 - d. Compute policy loss as $B_{st}^{CLIP}(\theta) = B_{st}[\min(\tilde{c}_{st}(\theta)\hat{X}_{st}, \text{clip}(c_{st}(\theta), 1 - \epsilon, 1 + \epsilon)\hat{X}_{st})]$
 - e. Compute value function loss using the mean squared error
 - f. Perform a gradient descent step on the policy and value function losses
5. Every ST1 steps, update the target policy $\pi^* \leftarrow \pi$
6. Reduce learning rate $\alpha \leftarrow \alpha - \alpha_d$

ALGORITHM A3: PPO Training Procedure.

Appendix D

Input:

Environment, replay buffer capacity (BUFFER_CAPACITY), initial policy parameters (θ_{policy}), initial Q-function parameters (θ_{Q1} , θ_{Q2}), initial value function parameters (θ_V), number of episodes (E), maximum time steps per episode (T), initial state (s_1) and preprocessing function (ϕ), learning rate (α), discount factor (γ), entropy coefficient (α_{entropy}), target network update frequency (τ), number of mini-batches (M), mini-batch size (N)

Output:

Trained policy parameters (θ_{policy}), trained Q-function parameters (θ_{Q1} , θ_{Q2}), trained value function parameters (θ_V)

Computation:

1. Initialize networks and buffers:
 - Set policy network π with parameters θ_{policy}
 - Set two Q-networks (qf1, qf2) with parameters θ_{Q1} and θ_{Q2}
 - Set value network vf with parameters θ_V
 - Set target networks for Q and V ($qf1_{\text{target}}$, $qf2_{\text{target}}$, vf_{target}) with the same parameters as qf1, qf2, and vf, respectively
 - Set replay buffer BD with capacity BUFFER_CAPACITY
2. For every episode e in $[1, \dots, E]$ do:
 - a. Initialize Episode: Set initial state s_1 and preprocess initial state $\phi_1 = \phi(s_1)$
 - b. For each iteration s_t in $[1, \dots, T]$ do:
 - i. Sample action: Sample action a_t from policy π using the state representation ϕ_{st}
 - ii. Execute action:
 - Execute action a_{st} in the environment and observed reward r_t and next state x_{st+1}
 - Select next state $s_{\{st+1\}}$ and preprocess $\phi_{\{st+1\}} = \phi(s_{\{st+1\}})$
 - iii. Store transition: Store transition $(\phi_{st}, a_{st}, r_{st}, \phi_{\{st+1\}})$ in replay buffer BD
 - iv. Sample mini-batch: Sample a training batch of experiences $(\phi_k, a_k, r_k, \phi_{\{k+1\}})$ from BD
 - v. Compute target value: Compute target value y_k using:

$$y_k = r_k + \gamma (\min(qf1_{\text{target}}(\phi_{k+1}, a_{k+1}), qf2_{\text{target}}(\phi_{k+1}, a_{k+1})) - \alpha_{\text{entropy}} \log \pi(a_{k+1} | \phi_{k+1}))$$
 - c. Update Q-networks:
 - Update Q-networks (qf1 and qf2) by minimizing the loss:

$$L_Q(\theta_{Q1}, \theta_{Q2}) = (1/N) \sum_k [(qf1(\phi_k, a_k) - y_k)^2 + (qf2(\phi_k, a_k) - y_k)^2]$$
 - d. Update value network:
 - Update value network vf by minimizing the loss:

$$L_V(\theta_V) = (1/N) \sum_k [vf(\phi_k) - \min_a (qf1(\phi_k, a) - \alpha_{\text{entropy}} \log \pi(a | \phi_k))]^2]$$
 - e. Update policy network:
 - Update policy network π by maximizing the objective:

$$L_\pi(\theta_{\text{policy}}) = (1/N) \sum [\alpha_{\text{entropy}} \log \pi(a_k | \phi_k) - \min_i (qf1(\phi_k, a_k), qf2(\phi_k, a_k))]$$
 - f. Perform gradient descent:
 - Perform gradient descent steps on the policy and value function losses
 3. Update target networks:
 - Every TARGET_UPDATE_FREQ steps, update target networks using

$$\theta_{qf1_{\text{target}}} \leftarrow \tau \theta_{qf1} + (1 - \tau) \theta_{qf1_{\text{target}}}$$

$$\theta_{qf2_{\text{target}}} \leftarrow \tau \theta_{qf2} + (1 - \tau) \theta_{qf2_{\text{target}}}$$

$$\theta_{vf_{\text{target}}} \leftarrow \tau \theta_{vf} + (1 - \tau) \theta_{vf_{\text{target}}}$$
 4. Adjust learning rate (if required): Optionally adjust learning rate α if needed.

ALGORITHM A4: SAC Training Procedure.

Data Availability Statement

The data that support the findings of this study are available on request from the corresponding author. The data are not publicly available due to privacy or ethical restrictions.

Conflicts of Interest

The authors declare no conflicts of interest

Funding

No funding was received for this manuscript.

References

- [1] L.-T. Zhao, Y. Wang, S.-Q. Guo, and G.-R. Zeng, "A Novel Method Based on Numerical Fitting for Oil Price Trend Forecasting," *Applied Energy* 220 (2018): 154–163, <https://doi.org/10.1016/j.apenergy.2018.03.060>.

- [2] D. Szarek, Ł. Bielak, and A. Wylomańska, "Long-Term Prediction of the Metals' Prices Using non-Gaussian Time-Inhomogeneous Stochastic Process," *Physica A: Statistical Mechanics and Its Applications* 555 (2020): 124659, <https://doi.org/10.1016/j.physa.2020.124659>.
- [3] T. Fang, C. Zheng, and D. Wang, "Forecasting the Crude Oil Prices with an EMD-ISBM-FNN Model," *Energy* (2022).
- [4] K. Drachal, "Forecasting the Crude Oil Spot Price With Bayesian Symbolic Regression," *Energies* 16, no. 1 (2022): 4, <https://doi.org/10.3390/en16010004>.
- [5] O. Orojo, J. Tepper, T. McGinnity, and M. Mahmud, "A multi-Recurrent Network for Crude Oil Price Prediction," *IEEE Symposium Series on Computational Intelligence (SSCI)* (2019).
- [6] L. Madziwa, M. Pillalamarri, and S. Chatterjee, "Gold Price Forecasting Using Multivariate Stochastic Model," *Resources Policy* 76 (2022): 102544, <https://doi.org/10.1016/j.resourpol.2021.102544>.
- [7] P. Zhang and B. Ci, "Deep Belief Network for Gold Price Forecasting," *Resources Policy* 69 (2020): 101806, <https://doi.org/10.1016/j.resourpol.2020.101806>.
- [8] M. Risse, "Combining Wavelet Decomposition With Machine Learning to Forecast Gold Returns," *International Journal of Forecasting* 35, no. 2 (2019): 601–615, <https://doi.org/10.1016/j.ijforecast.2018.11.008>.
- [9] P. Foroutan and S. Lahmiri, "Deep Learning Systems for Forecasting," *Financial Innovation* (2024).
- [10] Z. Yuan, "Gold and Bitcoin Price Prediction Based on KNN, XGBoost and LightGBM Model," *Highlights in Science Engineering and Technology* 39 (2023): 720–725, <https://doi.org/10.54097/hset.v39i.6635>.
- [11] L. Boongasame, P. Viriyaphol, K. Tassanavipas, and P. Temdee, "Gold-Price Forecasting Method Using Long Short-Term Memory and the Association Rule," *Journal of Mobile Multimedia* (2022): <https://doi.org/10.13052/jmm1550-4646.1919>.
- [12] A. Vidal and W. Kristjanpoller, "Gold Volatility Prediction Using a CNN-LSTM Approach," *Expert Systems with Applications* 157 (2020): 113481, <https://doi.org/10.1016/j.eswa.2020.113481>.
- [13] H. Jiang, W. Hu, L. Xiao, and Y. Dong, "A Decomposition Ensemble Based Deep Learning Approach for Crude Oil Price Forecasting," *Resources Policy* 78 (2022): 102855, <https://doi.org/10.1016/j.resourpol.2022.102855>.
- [14] M. Su, H. Liu, C. Yu, and Z. Duan, "A New Crude Oil Futures Forecasting Method Based on Fusing Quadratic Forecasting With Residual Forecasting," *Digital Signal Processing* 130 (2022): 103691, <https://doi.org/10.1016/j.dsp.2022.103691>.
- [15] J. Sun, P. Zhao, and S. Sun, "A New Secondary decomposition-reconstruction-ensemble Approach for Crude Oil Price Forecasting," *Resources Policy* 77 (2022): 102762, <https://doi.org/10.1016/j.resourpol.2022.102762>.
- [16] A. Gadhave, A. Gadhave, M. Dubey, and J. Kharade, "Gold Price Prediction Using Machine Learning," *International Journal of Environmental Engineering* 06, no. 05 (2022): <https://doi.org/10.55041/ijssrem15027>.
- [17] D. S. Sankar and H. Sahu, "Gold Prices Prediction Using Random Forest," *International Journal for Research in Applied Science and Engineering Technology* 11, no. 5 (2023): 4261–4264, <https://doi.org/10.22214/ijraset.2023.52605>.
- [18] K. Vamshi, K. Chakradhar, and S. Lonkala, *Price Prediction of Gold Using Random Forest* (IJARIIE, 2023).
- [19] Z. Alameer, M. A. Elaziz, A. A. Ewees, H. W. Ye, and Z. Jianhua, "Forecasting Gold Price Fluctuations Using Improved Multilayer Perceptron Neural Network and Whale Optimization Algorithm," *Resources Policy* 61 (2019): 250–260, <https://doi.org/10.1016/j.resourpol.2019.02.014>.
- [20] Z. Zhang, S. Zohren, and S. Roberts, "Deep Reinforcement Learning for Trading," *The Journal of Financial Data Science* 2, no. 2 (2020): 25–40, <https://doi.org/10.3905/jfds.2020.1.030>.
- [21] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning With a Stochastic Actor," in *Proceedings of the 35th International Conference on Machine Learning*, 2018.
- [22] Z. Zhang, S. Zohren, and S. Roberts, "Deep Reinforcement Learning for Trading," *The Journal of Financial Data Science* 2, no. 2 (2020): 25–40, <https://doi.org/10.3905/jfds.2020.1.030>.
- [23] H. Buehler, L. Gonon, J. Teichmann, and B. Wood, "Deep Hedging," *Quantitative Finance* 19, no. 8 (2019): 1271–1291, <https://doi.org/10.1080/14697688.2019.1571683>.
- [24] Y. Li, W. Zheng, and Z. Zheng, "Deep Robust Reinforcement Learning for Practical Algorithmic Trading," in *IEEE* (2019).
- [25] G. Jeong and H. Y. Kim, "Improving Financial Trading Decisions Using Deep Q-learning: Predicting the Number of Shares, Action Strategies, and Transfer Learning," *Expert Systems with Applications* 117 (2019): 125–138, <https://doi.org/10.1016/j.eswa.2018.09.036>.
- [26] Z. Gao, Y. Gao, Y. Hu, Z. Jiang, and J. Su, "Application of Deep Q-Network in Portfolio Management," in *5th IEEE International Conference of Big Data Analytics* (2020).
- [27] P. Liu, Y. Zhang, F. Bao, X. Yao, and C. Zhang, "Multi-Type Data Fusion Framework Based on Deep Reinforcement Learning for Algorithmic Trading," *Applied Intelligence* 53, no. 2 (2022): 1683–1706, <https://doi.org/10.1007/s10489-022-03321-w>.
- [28] J. Carapuço, R. Neves, and N. Horta, "Reinforcement Learning Applied to Forex Trading," *Applied Soft Computing* 73 (2018): 783–794, <https://doi.org/10.1016/j.asoc.2018.09.017>.
- [29] M. Massahi and M. Mahootchi, "A Deep Q-learning Based Algorithmic Trading System for Commodity Futures Markets," *Expert Systems with Applications* 237 (2024): 121711, <https://doi.org/10.1016/j.eswa.2023.121711>.
- [30] X. Wu, H. Chen, J. Wang, L. Troiano, V. Loia, and H. Fujita, "Adaptive Stock Trading Strategies With Deep Reinforcement Learning Methods," *Information Sciences* 538 (2020): 142–158, <https://doi.org/10.1016/j.ins.2020.05.066>.
- [31] D. Caldara and M. Iacoviello, *Measuring Geopolitical Risk* (FRB International Finance, 2018).
- [32] B. Kaur, B. K. Sidhu, and G. S. Bhathal, "The Influence of Geopolitical-Risk on Futures of Gold Commodity Market: A Machine Learning Perspective," in *Recent Trends in Intelligent Computing and Communication* (2025), 932.
- [33] B. Kaur, B. K. Sidhu, and G. S. Bhathal, "Reinforcement Learning Approach to Predictive Analytics," in *International Conference on Progressive Computational Intelligence, Information Technology, and Networking* (2025).